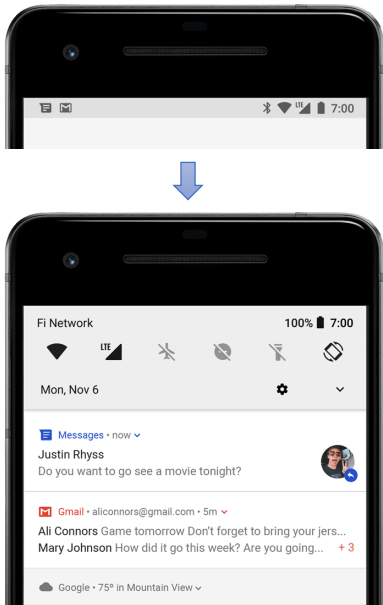


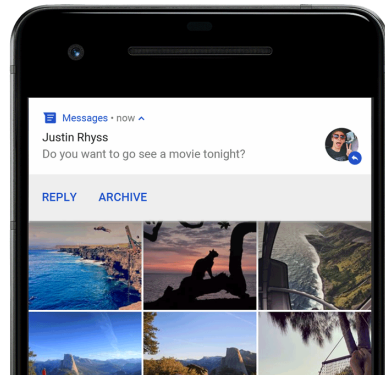
3. Notifications

- A notification is a message that Android displays outside an app's user interface to provide the user with reminders or other information
- There are different formats for Android notifications:

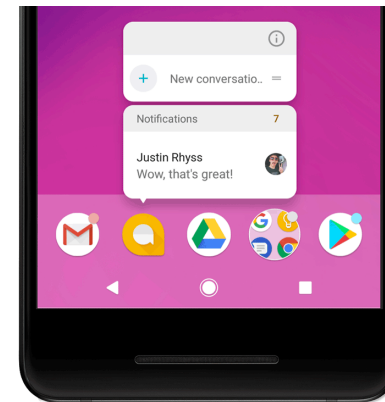
Status bar



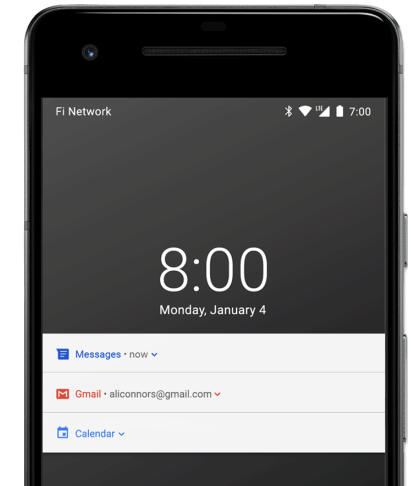
Heads-up notification



App icon badge



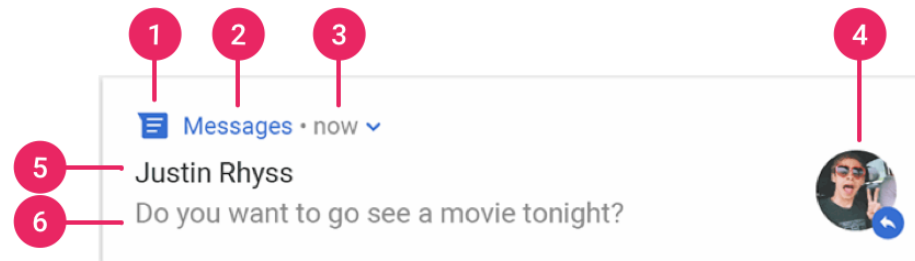
Lock screen



<https://developer.android.com/guide/topics/ui/notifiers/notifications>

3. Notifications - Anatomy

- The design of a notification has different elements:
 1. Small icon: required; set using `setSmallIcon()` in the notifications builder
 2. App name: provided by the system
 3. Time stamp: provided by the system. It can be overridden using `setWhen()`
 4. Large icon: optional; set using `setLargeIcon()`
 5. Title: optional; set using `setContentTitle()`
 6. Text: optional; set using `setContentText()`

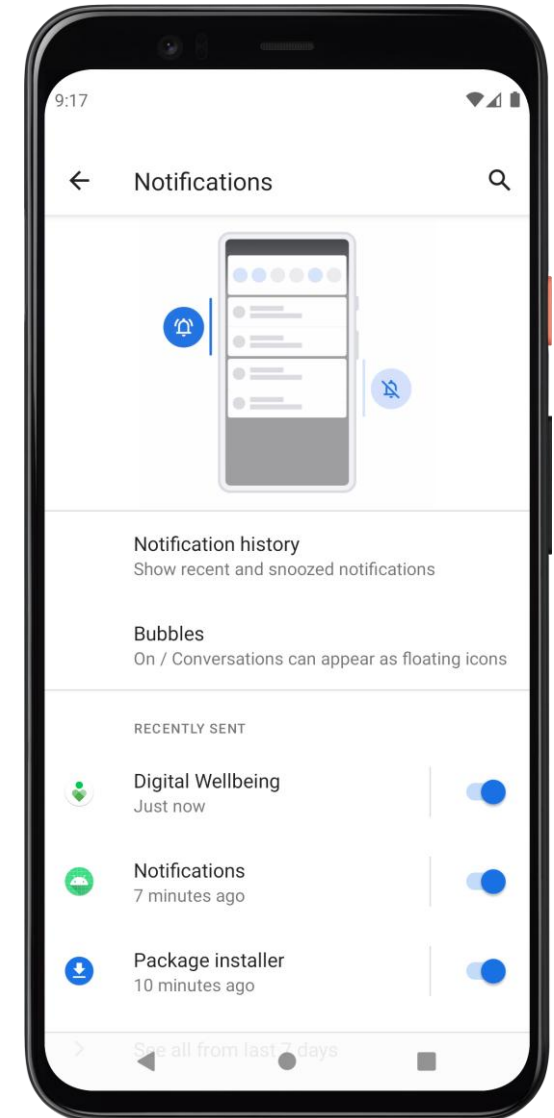


Nevertheless, some of these elements are not available in specific Android devices

<https://developer.android.com/guide/topics/ui/notifiers/notifications>

3. Notifications - Channels

- Starting in Android 8.0 (Oreo), all notifications must be assigned to a channel
 - A channel is a categorization mechanism that allows us to group notifications into different types based on their content, importance, or other factors
- Users can customize different aspects of notifications (Settings → App & notifications → Notifications)
 - For instance, users can disable notifications of specific apps



3. Notifications - Status bar

- The examples repository contains a project implementing different kinds of notifications

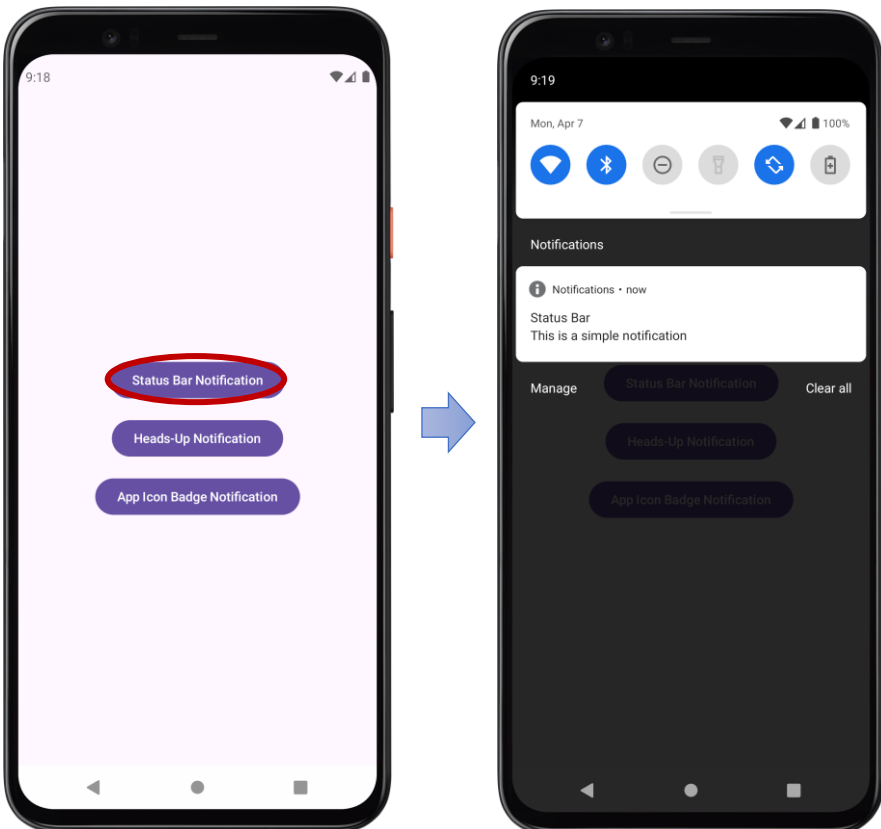
First, we create the notifications channels

The channel is composed by an identifier, a name, an importance level, and a description

```
private fun createNotificationChannels() {  
    // Create the NotificationChannel, but only on API 26+ because  
    // the NotificationChannel class is not in the Support Library  
    if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {  
        // Channel for standard notifications  
        val standardChannel = NotificationChannel(  
            STANDARD_CHANNEL_ID,  
            STANDARD_CHANNEL_NAME,  
            NotificationManager.IMPORTANCE_DEFAULT  
        ).apply {  
            description = STANDARD_CHANNEL_DESCRIPTION  
        }  
  
        // Channel for heads-up notifications  
        val headsUpChannel = NotificationChannel(  
            HEADS_UP_CHANNEL_ID,  
            HEADS_UP_CHANNEL_NAME,  
            NotificationManager.IMPORTANCE_HIGH // Required for heads-up  
        ).apply {  
            description = HEADS_UP_CHANNEL_DESCRIPTION  
        }  
  
        // Register the channels with the system  
        val notificationManager: NotificationManager =  
            context.getSystemService(Context.NOTIFICATION_SERVICE) as NotificationManager  
        notificationManager.createNotificationChannel(standardChannel)  
        notificationManager.createNotificationChannel(headsUpChannel)  
    }  
}
```

3. Notifications - Status bar

Then, at some point we launch the notifications



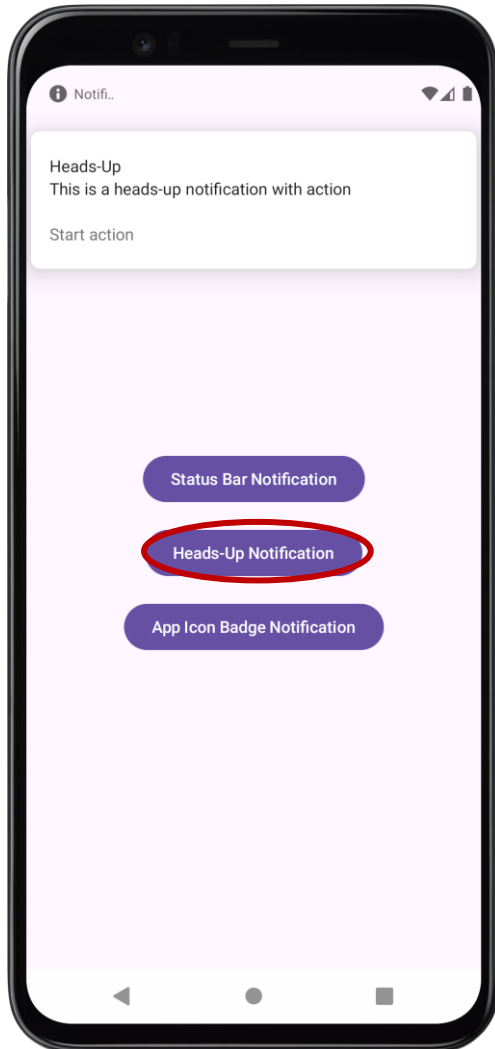
```
@SuppressWarnings("MissingPermission")
fun statusBarNotification(title: String, content: String) {
    val builder = NotificationCompat.Builder(context, STANDARD_CHANNEL_ID)
        .setSmallIcon(android.R.drawable.ic_dialog_info)
        .setContentTitle(title)
        .setContentText(content)
        .setPriority(NotificationCompat.PRIORITY_DEFAULT)

    with(NotificationManagerCompat.from(context)) {
        notify(NOTIFICATION_ID, builder.build())
    }
}
```

```
<uses-permission android:name="android.permission.POST_NOTIFICATIONS" />
```

This permission is required in the manifest

3. Notifications - Heads-up



```
@SuppressWarnings("MissingPermission")
fun headsUpNotification(title: String, content: String) {
    val builder = NotificationCompat.Builder(context, HEADS_UP_CHANNEL_ID)
        .setSmallIcon(android.R.drawable.ic_dialog_info)
        .setContentTitle(title)
        .setContentText(content)
        .setPriority(NotificationCompat.PRIORITY_HIGH)
        .setFullScreenIntent(null, true)
        .addAction(R.drawable.ic_launcher_foreground,
            context.getString(R.string.start_action), getPendingIntent())
        .setAutoCancel(true)

    with(NotificationManagerCompat.from(context)) {
        notify(NOTIFICATION_ID + 1, builder.build())
    }
}

fun getPendingIntent(): PendingIntent {
    val intent = Intent(Intent.ACTION_DIAL, "tel:666555444".toUri())
    return PendingIntent.getActivity(context, 0, intent, PendingIntent.FLAG_IMMUTABLE)
}
```

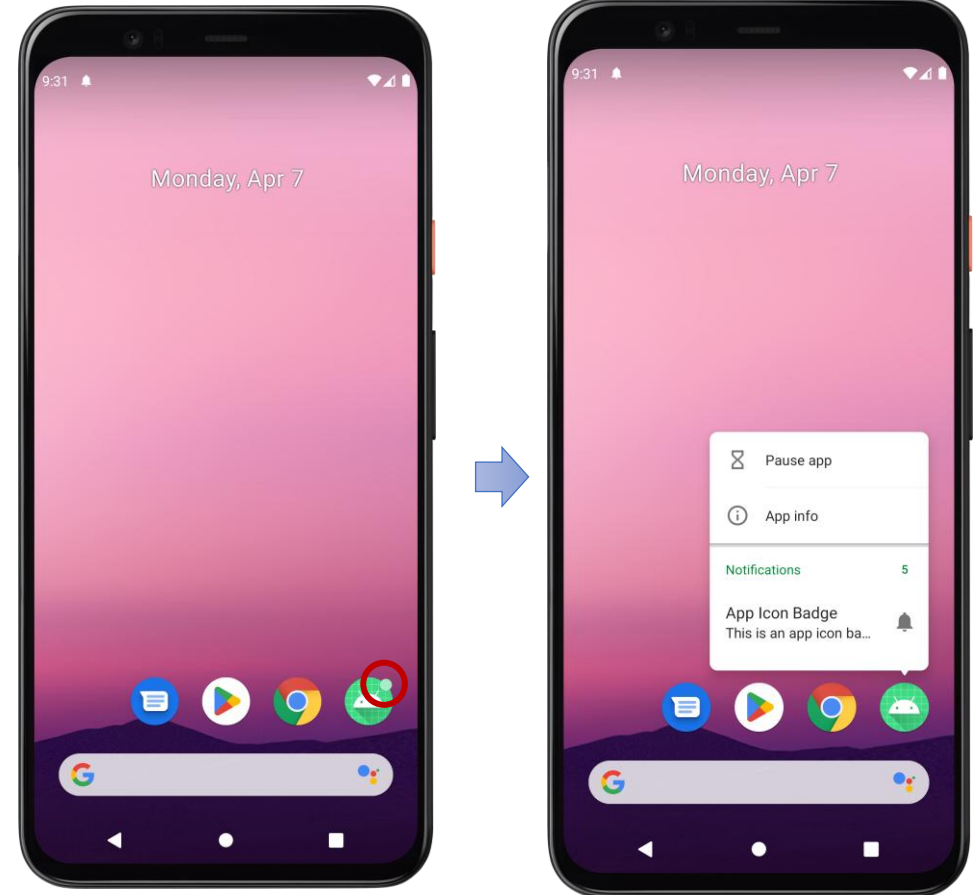
A PendingIntent is a token we give to a foreign app (e.g. NotificationManager) which allows this app to execute a given intent

3. Notifications - App icon badge

```
@SuppressWarnings("MissingPermission")
fun badgeNotification(title: String, content: String) {
    val notification = NotificationCompat.Builder(context, STANDARD_CHANNEL_ID)
        .setSmallIcon(R.drawable.baseline_notifications_24)
        .setContentTitle(title)
        .setContentText(content)
        .setPriority(NotificationCompat.PRIORITY_DEFAULT)
        .setContentIntent(getPendingIntent())
        .setAutoCancel(true)
        .setNumber(5) // This makes the badge appear
        .setBadgeIconType(NotificationCompat.BADGE_ICON_SMALL)
        .build()

    with(NotificationManagerCompat.from(context)) {
        notify(NOTIFICATION_ID + 2, notification)
    }
}
```

The app icon badge (also known as *notification dot*) is a visual indicator displayed on the app's icon to convey certain information or notifications to the user.



3. Notifications - Lock screen

- Notifications can appear on the lock screen as of Android 5
 - This feature can be useful, for example, in messaging apps
- To control the level of detail visible in the notification from the lock screen, call `setVisibility()` and specify one of the following values:
 - `VISIBILITY_PUBLIC`: the notification full content shows on the lock screen
 - `VISIBILITY_SECRET`: no part of the notification shows on the lock screen
 - `VISIBILITY_PRIVATE`: only basic information, such as the notification icon and the content title, shows on the lock screen. The notification full content doesn't show

We see an example of lock screen notification in the next section

Table of contents

1. Introduction
2. Services
3. Notifications
- 4. Alarms**
5. Takeaways